Modeling dynamic consonant-vowel (CV) coarticulation requires coupling three distinct acoustic mechanisms: a low-frequency voicing lead (voice bar) during occlusion, an aerodynamic transient release burst with place-dependent spectral tilt, and rapid nonlinear formant trajectories transitioning from consonant acoustic loci to vowel steady states.

```
      [ Closure / Voice Bar ]  ->  [ Transient Release Burst ]  ->  [ Formant Transitions ]  ->  [ Vowel Steac
Time:  t = 0 to 40 ms               t = 40 to 55 ms                 t = 55 to 110 ms            t > 110 ms
Source: Attenuated Glottal Wave      Shaped White Noise + Glottal    Periodic Glottal Pulse       Periodic Glc
Poles: Low-pass (~150 Hz)           Bandpass (Locus Resonator)      F1..F4 Moving Resonators     F1..F4 Static
```

Acoustic Mechanics & Locus Formulations

1. Acoustic Locus Equations

The starting frequencies of formant transitions do not originate at arbitrary values; they track virtual points of origin called **acoustic loci** (F_{locus}), determined by the point of oral constriction. The onset frequency $F_{2,\text{onset}}$ relates to the vowel target $F_{2,\text{vowel}}$ via empirical regression:

$$F_{2,\text{onset}} = k \cdot F_{2,\text{vowel}} + c$$

Formant trajectories follow an exponentially damped relaxation curve toward the vowel targets:

$$F_n(t) = F_n^{\text{vowel}} + \left(F_n^{\text{onset}} - F_n^{\text{vowel}}\right) e^{-\frac{t - t_{\text{rel}}}{\tau_n}}, \quad t \geq t_{\text{rel}}$$

where τ_n is the transition time constant (typically 15–35 ms), and $F_1(t)$ always begins near total occlusion ($150\text{--}250$ Hz) and rises rapidly as the vocal tract opens.

2. Release Burst Synthesis

The release burst is modeled as shaped Gaussian noise $w[n]$ filtered through a 2nd-order resonator centered at the stop's characteristic burst frequency F_{burst}, modulated by an asymmetric power envelope:

$$e_{\text{burst}}(t) = A_{\text{burst}} \left(\frac{t - t_{\text{rel}}}{t_{\text{rise}}}\right) \exp\left(1 - \frac{t - t_{\text{rel}}}{t_{\text{rise}}}\right), \quad t \geq t_{\text{rel}}$$

3. Stop Acoustic Parameters

Feature	Bilabial (/b/)	Alveolar (/d/)	Velar (/g/)
Constriction Point	Lips (diffuse-flat/falling)	Alveolar ridge (diffuse-rising)	Hard/soft palate (compact)
F_1 Locus	180 Hz	200 Hz	220 Hz
F_2 Locus	700–800 Hz	1700–1800 Hz	2200–3000 Hz (front) / 1200–1600 Hz (back)
F_3 Locus	2100 Hz	2700 Hz	2000–2400 Hz (*pinch* near F_2)
Burst Center (F_b)	500–1200 Hz	3500–4500 Hz	1800–2600 Hz
Burst Duration	5–10 ms	10–15 ms	15–25 ms
Voice Bar Amplitude	High (long closure)	Medium	Low/Short

Python Synthesis Engine

This implementation models time-varying cascade biquad resonators with sample-by-sample direct form II transposed filtering to eliminate boundary artifacts during high-rate formant excursions.

```python
import numpy as np
import scipy.signal as signal

class DynamicCVSynthesizer:
    def __init__(self, sr=44100):
        self.sr = sr

    def _rosenberg_glottal_pulse(self, f0, duration):
        """Generates a derivative Rosenberg glottal airflow excitation."""
        N = int(self.sr * duration)
```

```
        excitation = np.zeros(N)
        period = int(self.sr / f0)

        # Rosenberg glottal model parameters
        t_open = int(0.40 * period)
        t_close = int(0.16 * period)

        for p in range(0, N, period):
            for i in range(period):
                idx = p + i
                if idx >= N:
                    break
                if i < t_open:
                    excitation[idx] = 0.5 * (1 - np.cos(np.pi * i / t_open))
                elif i < t_open + t_close:
                    excitation[idx] = np.cos(np.pi * (i - t_open) / (2 *
                    t_close))
                else:
                    excitation[idx] = 0.0

        # Differentiate to simulate lip radiation impedance
        radiation = np.diff(excitation, prepend=0)
        return radiation / (np.max(np.abs(radiation)) + 1e-9)

    def _biquad_coeffs(self, f, bw):
        """Calculates digital second-order resonator pole coefficients."""
        f = np.clip(f, 20.0, self.sr * 0.49)
        bw = np.clip(bw, 10.0, self.sr * 0.25)
        r = np.exp(-np.pi * bw / self.sr)
        theta = 2.0 * np.pi * f / self.sr
        a1 = -2.0 * r * np.cos(theta)
        a2 = r * r
        b0 = 1.0 - r  # Normalized unity resonance gain
        return b0, a1, a2

    def _time_varying_biquad_cascade(self, x, f_trajectories, bw_trajectories):
        """Applies cascade formant filtering with continuous state
        interpolation."""
        num_formants, N = f_trajectories.shape
        y = np.copy(x)

        for k in range(num_formants):
            w1 = 0.0
            w2 = 0.0
            y_stage = np.zeros(N)
            for n in range(N):
                f = f_trajectories[k, n]
                bw = bw_trajectories[k, n]
                b0, a1, a2 = self._biquad_coeffs(f, bw)

                # Direct Form II Transposed structure
                x_n = y[n]
                y_n = b0 * x_n + w1
                w1 = -a1 * y_n + w2
                w2 = -a2 * y_n

                y_stage[n] = y_n
            y = y_stage
        return y

    def synthesize_syllable(self, consonant='d', vowel='a', f0=120.0,
    duration=0.35):
        N = int(self.sr * duration)
        t = np.linspace(0, duration, N, endpoint=False)

        # Temporal milestones
        t_closure = 0.045     # 45 ms occlusion
        t_burst = 0.012       # 12 ms burst
        t_transition = 0.040  # 40 ms transition
        t_rel = t_closure
        t_vot = t_rel + 0.008 # Voice onset time

```

```
        # Canonical Vowel Formant Targets (F1, F2, F3, F4) & Bandwidths
        vowels = {
            'a': {'F': [730, 1090, 2440, 3400], 'BW': [80, 90, 130, 200]},
            'i': {'F': [270, 2290, 3010, 3500], 'BW': [50, 100, 160, 220]},
            'u': {'F': [300, 870, 2240, 3400],  'BW': [60, 80, 110, 180]}
        }

        # Consonant Locus Definitions
        stops = {
            'b': {'locus': [180, 750, 2100, 3400],  'burst_f': 800,
            'burst_bw': 400, 'burst_amp': 0.15},
            'd': {'locus': [200, 1800, 2700, 3400], 'burst_f': 4000,
            'burst_bw': 600, 'burst_amp': 0.35},
            'g': {'locus': [220, 2400 if vowel == 'i' else 1400, 2300, 3400],
                  'burst_f': 2200 if vowel == 'i' else 1600, 'burst_bw': 300,
                  'burst_amp': 0.45}
        }

        v_target = vowels[vowel]
        c_spec = stops[consonant]

        # 1. Glottal Voice Source & Closure Voice Bar
        raw_glottal = self._rosenberg_glottal_pulse(f0, duration)
        glottal_source = np.zeros(N)

        # Voice Bar: Low-pass filtered glottal leak during occlusion
        sos_voice_bar = signal.butter(2, 180, 'lowpass', fs=self.sr,
        output='sos')
        voice_bar = signal.sosfilt(sos_voice_bar, raw_glottal) * 0.18

        for n in range(N):
            if t[n] < t_rel:
                glottal_source[n] = voice_bar[n]
            elif t[n] >= t_vot:
                # Glottal amplitude ramp-up post VOT
                voicing_gain = np.clip((t[n] - t_vot) / 0.015, 0.0, 1.0)
                glottal_source[n] = raw_glottal[n] * voicing_gain

        # 2. Transient Release Burst Generation
        burst_source = np.zeros(N)
        noise = np.random.normal(0, 1, N)
        sos_burst = signal.butter(2, [c_spec['burst_f'] - c_spec['burst_bw']/2,
                                      c_spec['burst_f'] + c_spec['burst_bw']/2],
                                  'bandpass', fs=self.sr, output='sos')
        filtered_noise = signal.sosfilt(sos_burst, noise)

        for n in range(N):
            if t_rel <= t[n] < t_rel + t_burst:
                tau_rise = t_burst * 0.2
                dt = t[n] - t_rel
                env = (dt / tau_rise) * np.exp(1.0 - dt / tau_rise)
                burst_source[n] = filtered_noise[n] * env * c_spec['burst_amp']

        total_excitation = glottal_source + burst_source

        # 3. Dynamic Formant Trajectories (F1-F4)
        f_trajectories = np.zeros((4, N))
        bw_trajectories = np.zeros((4, N))

        tau = t_transition / 2.8  # Decay constant to reach ~95% target in
        transition window

        for k in range(4):
            f_locus = c_spec['locus'][k]
            f_vowel = v_target['F'][k]
            bw_vowel = v_target['BW'][k]

            for n in range(N):
                if t[n] < t_rel:
                    f_trajectories[k, n] = f_locus
                else:
                    delta_t = t[n] - t_rel
```

```
                    # Non-linear asymptotic shift from locus to target
                    f_trajectories[k, n] = f_vowel + (f_locus - f_vowel) * np.
                    exp(-delta_t / tau)

                bw_trajectories[k, n] = bw_vowel

        # 4. Synthesize Syllable via Time-Varying Vocal Tract Resonators
        audio = self._time_varying_biquad_cascade(total_excitation,
        f_trajectories, bw_trajectories)

        # Normalize and remove DC offset
        audio = audio - np.mean(audio)
        audio = audio / (np.max(np.abs(audio)) + 1e-9)
        return audio
```

Implementation Details for Coarticulation Nuances

- **Velar Pinch Modeling:** When generating /g/ before front vowels (such as /gi/), F_2 and F_3 loci converge near $2300–2500\ \text{Hz}$. In the synthesizer, ensure the difference $|F_3(t) - F_2(t)|$ remains minimal during the first $25\ \text{ms}$ of the burst release before diverging toward vowel steady states.
- **Filter Stability Under Modulation:** Instantaneous changes in biquad a_1, a_2 coefficients can introduce high-frequency clicks. Direct Form II Transposed structures preserve state variables (w_1, w_2) across coefficient updates without requiring cross-fading buffers.
- **Aspiration / Voice Onset Asymmetry:** Unvoiced stops (/p/, /t/, /k/) require extending the open-glottis noise period (VOT to $40–80\ \text{ms}$) and applying an open-quotient tilt ($+12\ \text{dB/octave}$ attenuation on higher harmonics) to represent aspiration turbulence before full glottal pulse synchronization.